

Technical Report for Noodle Scholar



Xiaoshen Han^{*†}

522030910045

Huanyu Li^{*†}

522030910194

Xingyu Ji^{*†}

522030910210

Hao Guan^{*†}

522030910072

Rui Tao^{*†}

522030910024

January 6, 2024

^{*} Equal Contribution

[†] ICE2604 group 5

Abstract

This project develops "Noodle Scholar", a comprehensive academic search engine, which provides robust, efficient, and user-friendly access to a vast array of academic resources. We employed MongoDB for its efficient handling of over 2,000,000 papers, ensuring a scalable and flexible database structure. Our unique crawling techniques, Naive Crawler and Crawllee Crawler, efficiently gather metadata arXiv. For image parsing, we utilized PyMuPDF and the advanced foundation-model-based Grounding DINO to enhance processing capabilities, especially for non-embedded and scanned images. A significant innovation is our table parsing approach, where we introduced novel ways of both learning-based approaches and production-level choice, bypassing traditional OCR tools and providing a more efficient and accurate parsing of complex scientific tables. The frontend, built on Vue and Bootstrap, offers a responsive, user-friendly interface. The performance of our website has also been optimized. The backend, powered by Sanic, guarantees a robust and high-speed website experience. Through Elasticsearch, we've enabled a powerful search engine, improving user experience with fast and accurate retrieval capabilities.



Contents

1	Database	1
1.1	Choice of Database	1
1.2	Data of 100 PDFs	1
1.2.1	Source	1
1.2.2	Inserting into Database	1
1.3	Data of Arxiv Papers	2
1.3.1	Source	2
1.3.2	Data Processing	2
2	Crawler	4
2.1	Naive Crawler	4
2.1.1	Implementation	4
2.1.2	Efficiency	4
2.2	Crawlee crawler	5
2.2.1	A common method for crawlee	5
2.2.2	An easier way for arxiv	5
3	Backend	6
3.1	Backend Frame selection	6
3.2	Basic Functionality	7
3.3	Advanced Functionality	7
3.3.1	User Registration and Login	7
3.3.2	CAPTCHA Verification	8
3.3.3	Model summary and Q&A	8
3.4	Website Deployment	8

4	Image Parsing	8
4.1	Image Parsing with PyMuPDF	8
4.1.1	Intro of PyMuPDF	8
4.1.2	Implementation	9
4.1.3	Limitations and Problems	9
4.2	Image Parsing with Foundation Model	9
4.2.1	Why Foundation Model	9
4.2.2	Our Approach	9
5	Table Parsing	11
5.1	Primal attempt	11
5.2	Introducing Nougat	12
5.3	Towards production level deployment	12
6	Frontend	13
6.1	Framework	13
6.2	Performance Optimization	13
6.2.1	Asynchronous Issues with Mounting Function	13
6.2.2	Optimizing Reflow and Redraw Times	14
6.2.3	Layout Strategy	15
7	Searching Engine	16
7.1	Data Import	16
7.1.1	Select Appropriate Types for Index Construction	16
7.1.2	Enhance Search Speed	16
7.2	Search Functionality	16
7.2.1	Basic Features	16

7.2.2	Advanced Search Features	17
7.3	Processing Search Results	17
8	Features	17
8.1	Basic Features	18
8.2	Special Features	18
8.3	Art	19
9	Visualization	19



1 Database

1.1 Choice of Database

In this project, we choose MongoDB instead of MySQL as our database system. Table 1 shows the comparison between MongoDB and MySQL. The main reasons are as follows:

1. Data Storage Method: MongoDB employs BSON (Binary JSON) for data storage. This format offers greater flexibility compared to traditional relational databases, particularly in handling unstructured data.

2. Adaptive Data Structure: MongoDB's schema-less architecture is more conducive to our project's dynamic development process. This flexibility allows for easier modifications and adaptations of the database structure as the project evolves.

3. Performance in Read/Write Operations: MongoDB exhibits enhanced performance in read and write operations, particularly in single-document transactions. Since there are totally 2,000,000+ papers in our database, MongoDB's efficient handling of such operations makes it a superior choice for our requirements.

1.2 Data of 100 PDFs

1.2.1 Source

The metadata of 100 papers is given in `100_PDF_MetaData.json`, including `paper_id`, `author`, `year`, `DOI`, etc. The `.pdf` of each paper is stored under the directory `\100_PDF`.

1.2.2 Inserting into Database

Since the source document (`100_PDF_MetaData.json`) is in JSON format, The import into the database is made especially easy. The `.json` file is first imported into a dict variable, and then the contents of the dictionary are traversed to get metadata of each paper. Each metadata can be seen as a document and can be inserted to the database.

For images and tables in `.pdf`, storing all the images and table files into the database is a significant workload. Instead, we index them by storing the corresponding file addresses in the database on the server.

Comparison Item	MongoDB	MySQL
Database Type	Non-relational, document-oriented	Relational, table-oriented
Storage Format	Binary JSON, more flexible structure	Fixed table and column structure
Query Language	Dynamic, object-oriented query language	Structured Query Language (SQL), more stringent in structure
Scalability	Distributed architecture, easy to expand	Centralized architecture, scaling can be complex
Consistency Model	Consistent, with eventual consistency in distributed scenarios	Strong consistency, ACID compliance
Transaction Support	Support for multi-document transactions	Support for complex transactions
Application Scenario	Suitable for big data, real-time analytics, IoT	Suitable for stable and complex enterprise applications
Cloud Service	Provides cloud service, MongoDB Atlas	Various cloud service providers offer MySQL
License	Open source, more permissive	Open source, more restrictive in commercial scenarios
Development Language	More flexible, supports a variety of programming languages	Generally uses SQL, more standardized

Table 1: Comparison between MongoDB and MySQL

1.3 Data of Arxiv Papers

1.3.1 Source

From <https://arxiv.org/>, we can easily crawl the metadata and pdf files of papers. In the next section, we will introduce our crawler in detail.

Due to time constraints, we utilizes an existing arxiv paper dataset which contains 2,000,000+ papers to realize other functions of our website.

1.3.2 Data Processing

Since the dataset is a huge `.json` file, we implemented read-by-line method to handle the data processing task.

Data processing includes following contents:

1. Authors

The "author" field in the dataset is managed in a strange way, we reformed the author information to a list that can be easily read.

Also, we count the number of papers each author publishes.

2. Publication Year

We extracted publication year of each paper from the "date" field and counted how many papers are published in each year.

3. Full Name of Subjects

Due to the use of abbreviations for the field labels in arXiv papers, we have created a mapping between these abbreviations and their corresponding full names.

4. Statistic Processing

Since every paper in arxiv has a domain (or subject), we counted the number of papers each main domain (or main subject) contains and its subdomain contains.

To improve the efficiency, we constructed a `.json` file to store domain related information, whose structure is shown below:

```
1 {
2   "main_domain1_abbr": [
3     {
4       "sub_subject": sub domain1 of main domain1,
5       "full_name": full name of the sub domain,
6       "num_papers": number of papers it contains
7     },
8     {
9       "sub_subject": sub domain2 of main domain1,
10      "full_name": full name of the sub domain,
11      "num_papers": number of papers it contains
12    },
13    ...
14  ],
15  "main_domain2_abbr": [
```

```

17         {
18             "sub_subject": sub domain1 of main domain2,
19             "full_name": full name of the sub domain,
20             "num_papers": number of papers it contains
21         },
22         {
23             "sub_subject": sub domain2 of main domain2,
24             "full_name": full name of the sub domain,
25             "num_papers": number of papers it contains
26         },
27
28         ...
29     ],
30
31     ...
32 }

```

2 Crawler

2.1 Naive Crawler

2.1.1 Implementation

Through the homepage of <https://arxiv.org/>, we can easily crawl all the domains and store them.

Entering one of the domains (e.g. CS, <https://arxiv.org/list/cs/recent>, we can crawl the url of each paper published recently. For each url, we then crawl the metadata of the paper.

2.1.2 Efficiency

Through testing, the efficiency is about 6000 papers per hour and it is far from our goal. Therefore, we try to use another method.

2.2 Crawllee crawler

Crawllee is a web scraping and browser automation library. It can be used as a powerful crawler to grasp information in the Internet. The development logic of using this library is very reasonable. We only need to choose the entrance url, tell the crawllee what urls he should enqueue, and tell him what data he should save. There are 3 main crawler classes available for use in Crawllee: CheerioCrawler, PuppeteerCrawler and PlaywrightCrawler. CheerioCrawler is the simplest crawler. He will not render the JavaScript so we can use it in the same way as we use in the normal crawler. PlaywrightCrawler and PuppeteerCrawler will render the JavaScript automatically, so we can get the same web page as accessed through the browser. We will elaborate on two ideas using PlaywrightCrawler for crawling Arxiv below.

2.2.1 A common method for crawllee

We choose [Arxiv](#) as the entrance url. And we will find there are many subjects and its hrefs in this page, and then we enqueue all the useful urls (using css selector as filter). Then we will go to the page like [this one\(https://arxiv.org/list/astro-ph.GA/recent\)](https://arxiv.org/list/astro-ph.GA/recent) and when we click all we will go to [this page\(https://arxiv.org/list/astro-ph.GA/pastweek?show=63\)](https://arxiv.org/list/astro-ph.GA/pastweek?show=63). The difference between the two urls is quite clear, so our crawler doesn't need to go to the former urls in fact but we can edit it to go to the latter one directly.(We can also change the parameter show to a quite large number so as to get all papers we can get) Then we enqueue all urls like [this one\(https://arxiv.org/abs/2401.02328\)](https://arxiv.org/abs/2401.02328). And at last we can get all the information we need in this paper.

2.2.2 An easier way for arxiv

We can easily find that the links to papers on Arxiv are regular. Just like the string of numbers 2401.02328, 24 represents the year 2024, 01 represents January, and 02328 is the 02328th article. We can simply traverse to get all urls. Since we don't know what the cut-off sequence number is, we can let the crawler crawl one article before adding the next one to the queue, and stop joining the queue until a certain article cannot crawl any useful information.(For URLs that are guaranteed to exist, joining the queue all at once in advance will help speed up crawling, so this method of writing is actually used in the code.)

3 Backend

3.1 Backend Frame selection

After investigation, it has been found that the Sanic framework based on Python has the following advantages:

1. **Significant Advantages in Asynchronous Processing Capabilities** As demonstrated in the figure 1(a), Sanic shows outstanding performance in terms of Queries Per Second (QPS), far surpassing other comparative frameworks. This data indicates that Sanic, utilizing Python's asynchronous programming features, effectively enhances concurrent processing capabilities through non-blocking IO operations and event loops. In facing a large number of parallel requests, Sanic maintains high throughput and stability of service, which is crucial for modern Web applications that need to handle large-scale user interactions.

2. **Minimization of Response Latency** Further analysis of the latency charts shows that Sanic also excels in terms of delay. (Shown in figure 1(b)) Low latency means faster response times, which directly impacts user experience and real-time data processing. Sanic's lightweight architecture and efficient request processing workflow significantly reduce the time required to handle each request, which is particularly important in building high-performance APIs.

3. **Resource Utilization Efficiency** Analysis of resource utilization rates reveals Sanic's optimizations in interacting with databases. The charts show that Sanic demonstrates high resource utilization efficiency, whether using MySQL or Redis. This indicates that the Sanic framework can effectively manage backend resources without sacrificing performance, reducing unnecessary resource wastage.

4. **Lightweight Design of the Framework** The overall performance assessment shows that Sanic's design philosophy leans towards being lightweight and efficient. This is not only reflected in performance metrics but also in its straightforward API and easy-to-understand programming patterns. This enables developers to quickly get started, accelerating the development cycle while maintaining code maintainability and scalability.

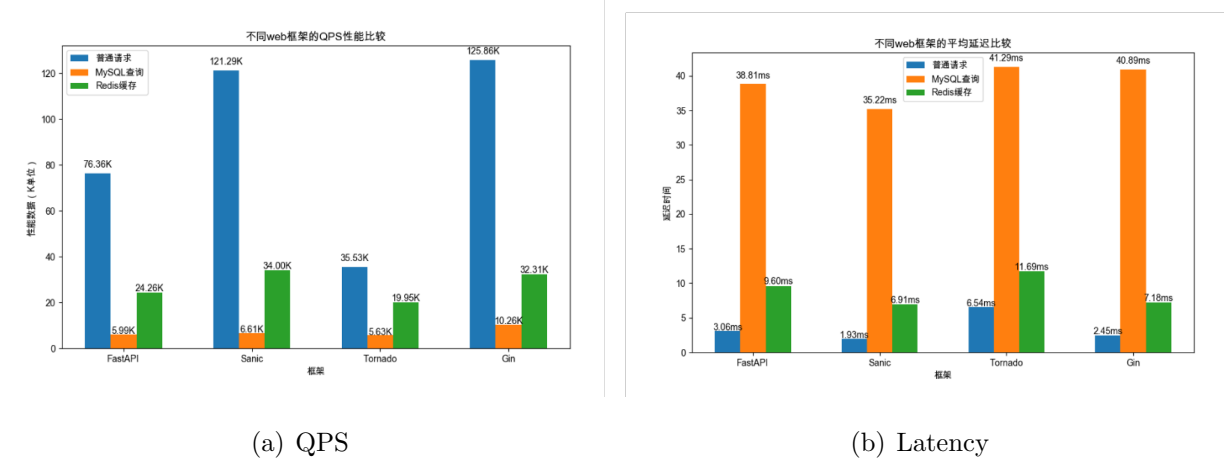


Figure 1: comparison sanic between fastapi

3.2 Basic Functionality

Utilizing the Sanic framework and guided by the principle of modular and open design, the backend code is designed to be as decoupled as possible. The basic functionality primarily encompasses three major parts: paper operations, search interface calls, and visualization-related code. Each part includes several specific interfaces, details of which can be referred to in the code.

3.3 Advanced Functionality

The backend implements user registration and login, CAPTCHA verification, and large model summary features.

3.3.1 User Registration and Login

User passwords are sensitive information. To protect this, we encrypt passwords using AES CFB mode and store the ciphertext. Additionally, we have implemented differentiation of user permissions. Considering the significant server load due to PDF downloads on the actual website, only logged-in users are allowed to download PDFs. Moreover, the model summary feature, which incurs higher costs, is available only to users with special permissions.

3.3.2 CAPTCHA Verification

Following the implementation of user registration and login, we recognized that allowing registration with just a username and password, even with limits on the download quantity to prevent malicious requests and relieve server pressure, would still enable mass malicious registrations through scripts, bypassing restrictions. To counter this, we considered implementing CAPTCHA verification to limit some of these malicious registrations. We chose Cloudflare’s Turnstile technology, a common and mature industry solution, for this purpose.

3.3.3 Model summary and Q&A

With the continuous development of large model technology, there has been a significant leap in models’ understanding of natural language, including academic papers. We are considering integrating large model technology to summarize paper content and implement simple Q&A. This would enable users to quickly and easily understand the content of papers, thereby helping them to select articles that meet their needs.

3.4 Website Deployment

For production environments, using `npm run dev` for front-end development is not suitable. Therefore, we employ `sanic` as the production environment server for deploying the website, whose advantages have been mentioned before(in 1). Additionally, we have deployed the website on the public internet at [Noodle Scholar](#), using the cloudflare tunnel technology. This enables the website to be accessible online.

4 Image Parsing

4.1 Image Parsing with PyMuPDF

4.1.1 Intro of PyMuPDF

PyMuPDF is a high-performance Python library used for extracting, analyzing, converting, and manipulating data from PDF (and other) documents.

4.1.2 Implementation

We iterate through each page of a PDF file, extract the images on that page, and save them to a folder. Each PDF corresponds to a separate image folder.

4.1.3 Limitations and Problems

Some PDFs are scanned documents, which causes PyMuPDF to be unable to properly parse these 'pictures within pictures'. We need additional methods to handle these situations

4.2 Image Parsing with Foundation Model

4.2.1 Why Foundation Model

Following our discussion on PyMuPDF's limitations, especially with non-embedded and scanned images in PDFs, it becomes apparent that an alternate approach is necessary. Traditional deep learning methods, though effective, require extensive labeled datasets. This process is time-consuming and resource-intensive, unsuitable for our project's needs.

Turning our attention to foundation models, these advanced AI systems are pre-trained on diverse and extensive datasets. Their primary advantage is their adaptability to various tasks with minimal additional training. This feature is particularly beneficial for the image parsing part of our project.

In contrast to traditional methods that demand large volumes of specific training data, foundation models operate effectively with smaller datasets. This efficiency is a major benefit, allowing us to achieve desired outcomes without the need for extensive data collection and labeling.

4.2.2 Our Approach

In our project, we utilize Grounding DINO, a zero-shot object detection model. Grounding DINO is distinctive for its ability to detect objects not included in its training set. This model combines the DINO (a transformer-based detection method) and GLIP (focused on phrase grounding), allowing it to understand both language and visual content.

Grounding DINO's architecture includes a text backbone (like BERT) and an image

Segment Anything

Alexander Kirillov^{1,2,4} Eric Mintun² Nikhila Ravi^{1,2} Hanzi Mao² Chloe Rolland³ Laura Gustafson³
Tete Xiao³ Spencer Whitehead Alexander C. Berg Wan-Yen Lo Piotr Dollár⁴ Ross Girshick⁴
¹project lead ²joint first author ³equal contribution ⁴directional lead
Meta AI Research, FAIR

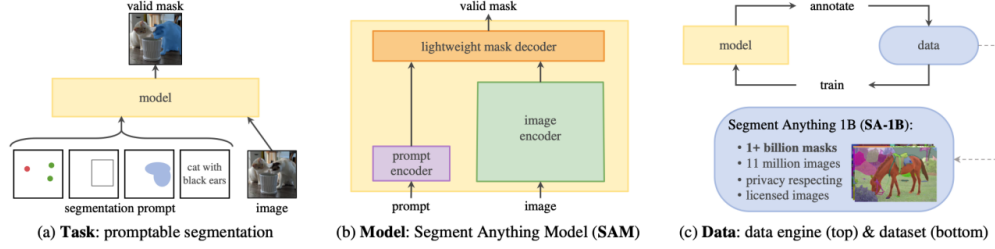


Figure 2: Famous Vision Foundation Model: Segment Anything

backbone (like Swin Transformer) for feature extraction. It also features a feature enhancer for cross-modality feature fusion and a cross-modality decoder, which combines image and text features for effective object detection.

This model is especially useful for tasks where flexibility is important, such as automatic data annotation or replacing heavy models. Grounding DINO simplifies image and video processing pipelines by enabling detection relationships to be analyzed using language constraints rather than complex components.

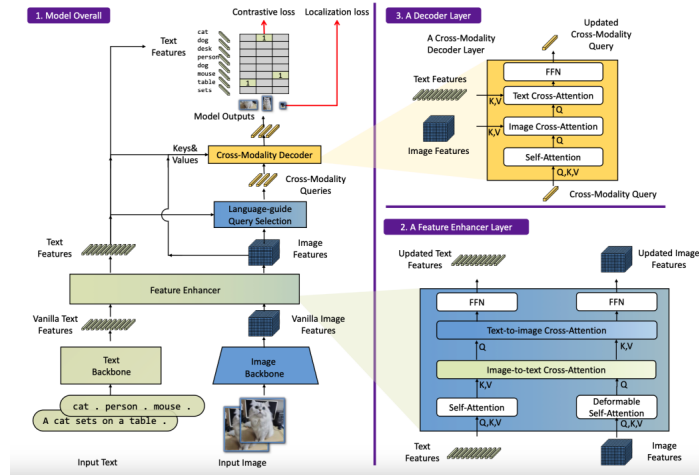


Figure 3: Structure Of Grounding DINO

We employed a really simple phrase prompt "graph", yet the result turns out to be good enough. All the scanned PDFs and non-embedded images are successfully extracted. Below is the demonstration of one result.

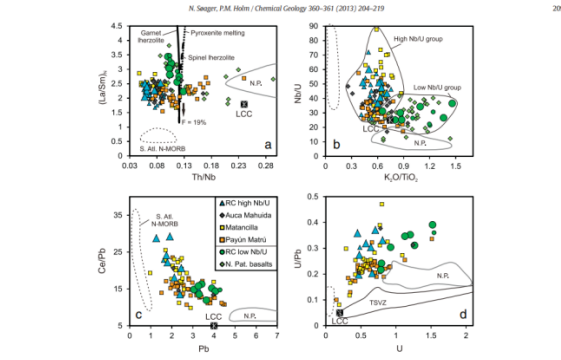


Fig. 3. a) $(La/Sm)_N$ vs Th/Nb . Normalisation values from McDonough and Sun (1995). b) Nb/U vs K_2O/TO_2 . c) Ce/Pb vs Pb . d) U/Pb vs U . The RC (Rio Colorado) samples fall in two groups with high and low Nb/U respectively. N.P. fields show the northern Pyrenean data from the Herault volcanic field with $Ba/Th < 125$ which are regarded as least contaminated (Seager et al., 2013). LCC is the lower continental crust average from Rudnick and Gao (2003). The melting paths in a) show aggregated non-modal batch melting of spinel (dashed line) and garnet (black line) Basaltic with a primitive mantle composition (McDonough and Sun, 1995). Mantle and melting modes are from McKenzie and O'Hara (1995). Distribution coefficients are from Kennedy et al. (1968) (olivine and orthopyroxene), Johnson (1968) (clinopyroxene and garnet), Salters and Longhi (1999) (Th in clinopyroxene and garnet) and Ellsworth et al. (2000) (spinel). The dotted line shows modal fractional melting of garnet pyroxene with a primitive mantle composition (McDonough and Sun, 1995) and with modal composition (82% clinopyroxene and 18% garnet) and distribution coefficients from Fretwell et al. (2004). Degree of melting (F) increases with decreasing $(La/Sm)_N$ up to $F = 19\%$. Larger RC and Payún Matrú symbols are data from Seager et al. (2013). No literature data from Payún Matrú are shown for clarity. Small RC symbols and Acaia Mahuida data are from Bernotti et al. (2009), Jacques et al. (2011) and Kay et al. (2004, 2013). Matancilla data are from (2013a, 2013b). North Patagonian basalts: Moudoulet et al. (2006), Molner and Stern (1988), Vankamp et al. (2010). TSVZ arc: Jacques et al. (2013). For datasets without Nb measurements, Nb has been calculated as $Ta \times 18$ which is the average Nb/Ta ratio of the RC basalts in the dataset of Seager et al. (2013). Data sources for South Atlantic N-MORB ($(La/Sm)_N < 1$) are the same as in Fig. 2.

from Patagonia (e.g. Conceição et al., 2005; Bjerg et al., 2009; Dantas et al., 2009) and northern Argentina (Lucassen et al., 2005) is the low contents of Ba and Rb relative to Th and U resulting in often very low Ba/Th ratios ($Ba/Th \approx 22$ in Agua Poca xenolith from Conceição et al., 2005). Since the Ba/Th ratio is largely unaffected by melting processes except for very low degree melts ($F < 1\%$) (Stracke and Bourdon, 2009) this type of lithospheric mantle would not be able to generate

the RC basalts which all have $Ba/Th > 100$. If instead the lithospheric mantle fragments were enriched in Ba but then they would also be expected to be enriched in Pb and U and have low Ce/Pb and Nb/U which the RC basalts are not. However, lithospheric mantle xenoliths display a wide variety of trace element compositions and it is difficult to draw unambiguous conclusions on this basis.

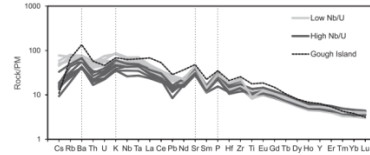


Fig. 4. Primitive mantle-normalized trace element patterns of the high and low Nb/U RC samples from Seager et al. (2013) which show the difference in trace element contents between the two groups. The two low Nb/U samples with $Ba/Nb > 17$ are not plotted. A sample from Gough Island (G131, Wilford and Stracke, 2006) has been shown for comparison. Normalisation values from McDonough and Sun (1995).

Figure 4: One page of a scanned PDF

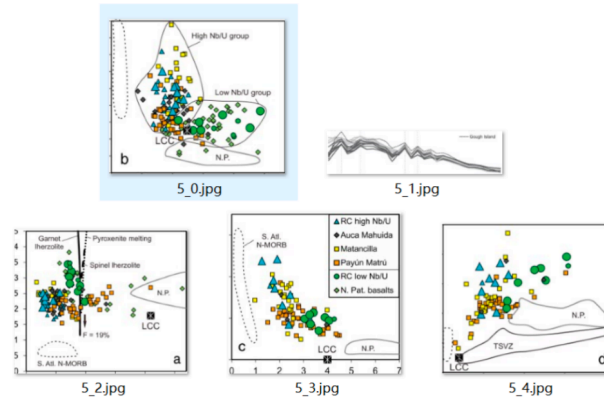


Figure 5: Our result of image parsing

5 Table Parsing

5.1 Primal attempt

We started with a method based on deep learning, given to us by our assistant teacher. It works well most of the time, demonstrating the effectiveness of deep learning method. But, we've hit some tricky cases where this method doesn't work as well. There are two ways to

fix these problems: either by manually figuring out solutions for each case or by making the deep learning model better with more data. Both ways take a lot of work and don't really fit our project. The first way never ends because new tricky cases keep coming up as we use more papers. The best way is to use the power of the neural network itself to handle these problems. This is more practical for us and helps solve our issue better.

5.2 Introducing Nougat

In our project, we chose to utilize Nougat, a more straightforward and self-reliant approach compared to the initial deep learning method. This decision was influenced by Nougat's ability to directly interpret the images of documents, making it highly adaptable to various formats encountered in academic papers. Unlike traditional methods that depend on external OCR tools, Nougat simplifies our process by internally handling the conversion of complex scientific texts into a structured format. This is particularly beneficial for our task of extracting tables, as it requires a robust system capable of understanding the intricate layouts and contents of academic documents. Its self-contained nature aligns with our project's need for a versatile and efficient solution, providing a more direct and less labor-intensive path to achieving our goal.

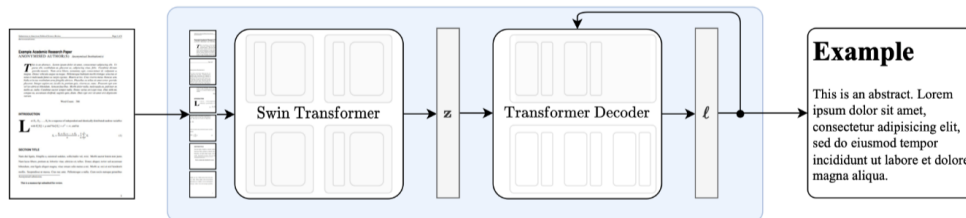


Figure 6: Network architecture of Nougat

5.3 Towards production level deployment

For a robust, production-level deployment, we suggest considering Adobe PDF Services, specifically the Adobe PDF Extract API and Document Generation API. These tools offer a high-accuracy and platform-agnostic solution, capable of extracting all PDF elements and generating documents with dynamic data. Their ability to quickly and accurately parse text, tables, and images, while maintaining format and structure, makes them ideal for real-time processing and professional-grade deployments.

6 Frontend

6.1 Framework

The frontend of our search engine is built upon **Vue** and **BootStrap**. Generally speaking, our website contains three main parts: Home, search results, paper detail, as shown below.

To connect all the pages, and further more, to pass parameters between these pages, we use **Router** to link and resolve each page.

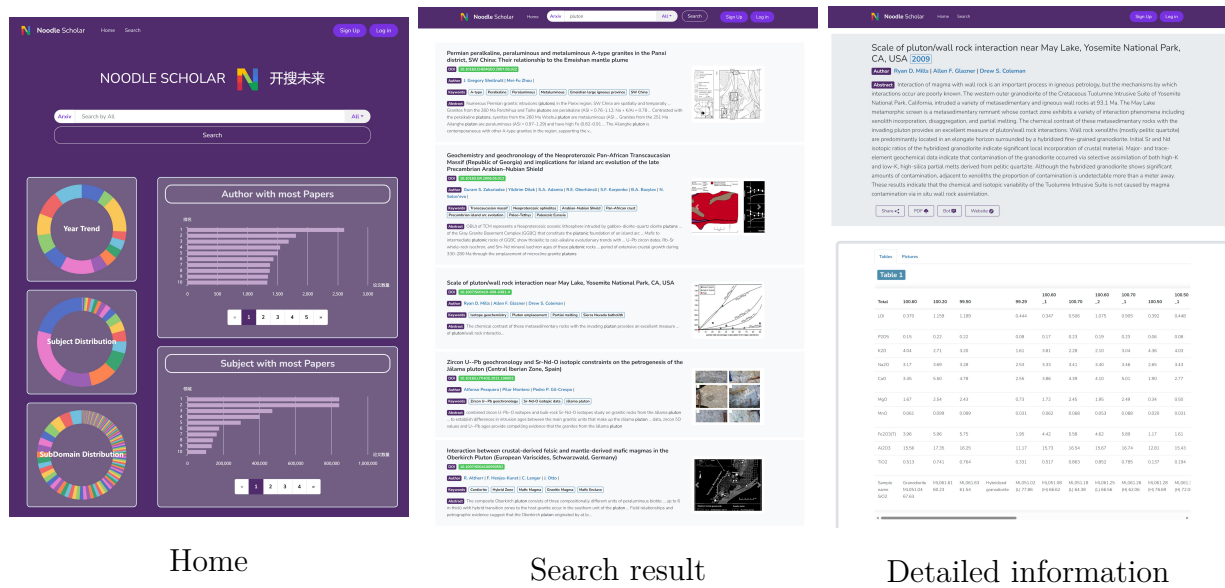


Figure 7: Overview of Noodle Scholar's website

6.2 Performance Optimization

Because of our huge amount of data, the execution time for searching and communicating cannot be ignored. Therefore, it's of great importance to improve website's performance, especially in sending requests and rendering elements.

6.2.1 Asynchronous Issues with Mounting Function

In the mounting function, Javascript executes all functions containing Axios HTTP requests synchronously. What's more, Javascript in the browser is single-threaded, so that there is no provision for thread locks.

This implies that element binding or other queries may be carried out before the HTTP

request has been fully processed with a high probability. To ensure that all requests are locked, we use Promise and asynchronous declaration.

On the one hand, declare the mounting function as an asynchronous function, and also add await declaration before all functions that need to be completed first, namely the functions that need to be locked. On the other hand, return all the Axios requests as Promise type.

6.2.2 Optimizing Reflow and Redraw Times

Figure 9 below shows the rendering process of the DOM tree and CSSOM tree by the browser.

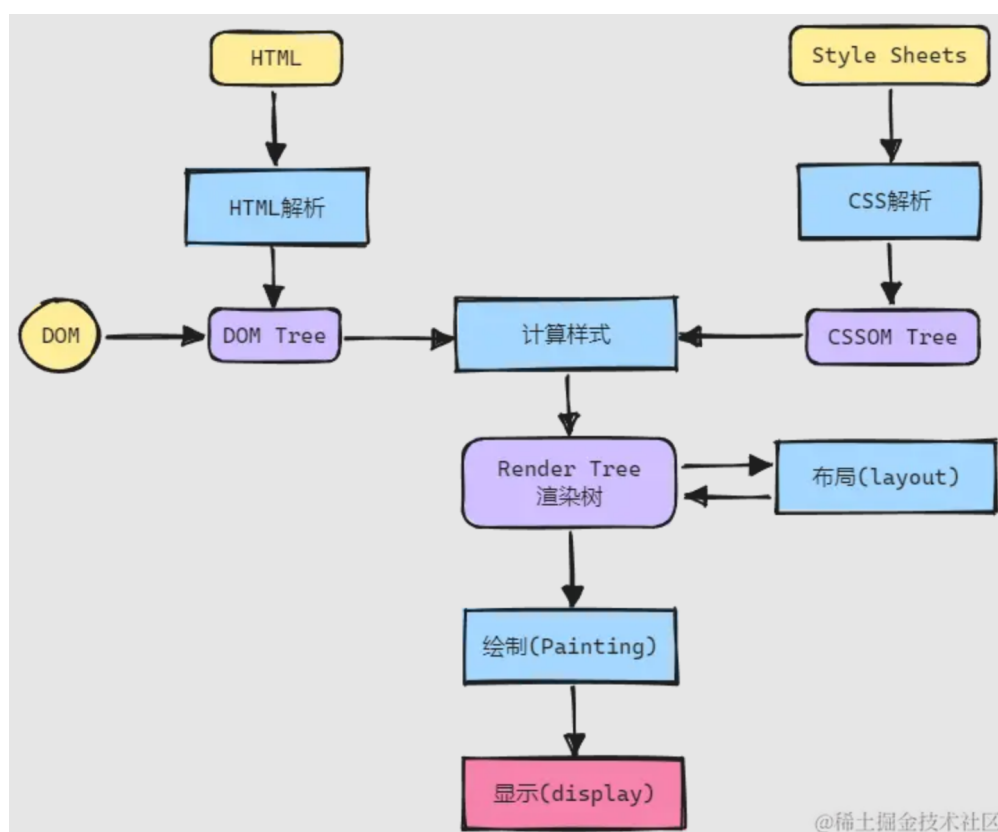


Figure 8: Process of rendering

Reflow and Repaint are the two main process how the web pages deal with elements' changes.

Reflow is a performance-consuming operation. The browser needs to re-calculate the geometric attributes of the elements and re-layout the entire page. So frequent reflows should be avoided as much as possible.

Redrawing does not affect the layout, it only re-draws the appearance of the elements, so the performance overhead is smaller than reflow.

Web pages will trigger reflows in some situations, such as the following three aspects:

1. The first rendering of the page (for example: refresh)

This part of the reflow, that is, the first rendering of the page, is unavoidable. We can only use limited methods(such as v-cloak) to prevent it from displaying uncompiled original data and eliminate the flicker problem during rendering.

2. Add or remove **visible** DOM elements

Considering our large amount of papers and visualization data, this is where we need to mainly consider the performance consumption of reflow. For example, the page-turning effect of the bar chart on the homepage. In order to prevent multiple HTTP requests caused by page turning, we make the visualization data **offline processed**, so every time we turn the page, it's just **modifying and copying and replacing the visibility** of the DOM element, so it will not trigger a page reflow.

3. Activate CSS pseudo-classes

Pseudo-classes, such as ":hover", ":active", ":focus" can result in reflowing of the page. We address this issue by rewriting all the activating process into JavaScript functions, so that the style can be changed simply by conditional judgement.

Taking everything into account, we converted all potential reflows conditions into redraws, which results in a remarkable decrease in time consumption.

6.2.3 Layout Strategy

We have made screen size adaptability adjustments to the "search results" page and the "paper detail" page. All elements are wrapped in several containers of **div** layers, which enables the inner elements to be flexibly adjusted.

Some of these containers use "fixed" and "relative" for position fixing, which allows the peripheral elements to **flexibly adapt to the page layout**.

Other containers use "absolute" for position fixing, which makes the inner elements **divorced from the document flow**. When the style is modified, only the element itself and its child elements will trigger a redraw, reducing the scope of the redraw.

7 Searching Engine

7.1 Data Import

7.1.1 Select Appropriate Types for Index Construction

During the import process, we achieved fuzzy and precise searching by selecting appropriate types for index construction. Fields like 'abstract' and 'title' were set to the 'text' type to facilitate fuzzy searches during queries, while fields such as 'doi' were designated as the 'keyword' type for precise querying. Additionally, fields like 'year' and 'date' were set as 'date' type, enabling us to limit the search to specific publication date of papers.

7.1.2 Enhance Search Speed

Initially, there was a conflict between the paper ID in the data and the ID in Elastic Search, so we used the 'paper_id' to distinguish different papers. This approach made backend queries in the database convenient. However, we later observed that using 'paper_id' for database requests entailed checking all articles, which costs lots of time. To enhance search speed, we later opted to use a hashed ID (named 'hash_id' in the database of Elastic Search) in the database during the import process. This change allowed the backend to efficiently locate data. Post-implementation, tests revealed that the time for backend data requests dropped dramatically from 18 seconds to 0.014 seconds, improving the website's performance by three orders of magnitude.

7.2 Search Functionality

We have achieved a variety of search functionality through two main approaches.

7.2.1 Basic Features

In terms of search capabilities, we have implemented domain-specific searches for author, title, abstract, subject, and DOI, as well as comprehensive searches across all these fields, in both the ArXiv database and the articles provided by the teaching assistants.

7.2.2 Advanced Search Features

We have also developed advanced search functionalities, including year range limitation searching, wildcard search, prefix search, and regular expression search.

Year Range Limitation Search Elasticsearch accepts two integer variables, `start__year` and `end__year`, to limit the publication year range of the papers in the search results.

Other Search Methods The wildcard and other search features are primarily enabled by changing the `query__type`. The default search method is `match`.

7.3 Processing Search Results

Highlighting Search Results For matched fields in search results, we enhance visibility by enclosing the content with HTML tags such as `< strong >< /strong >` before returning it to the frontend. This approach facilitates easier identification of matched fields.

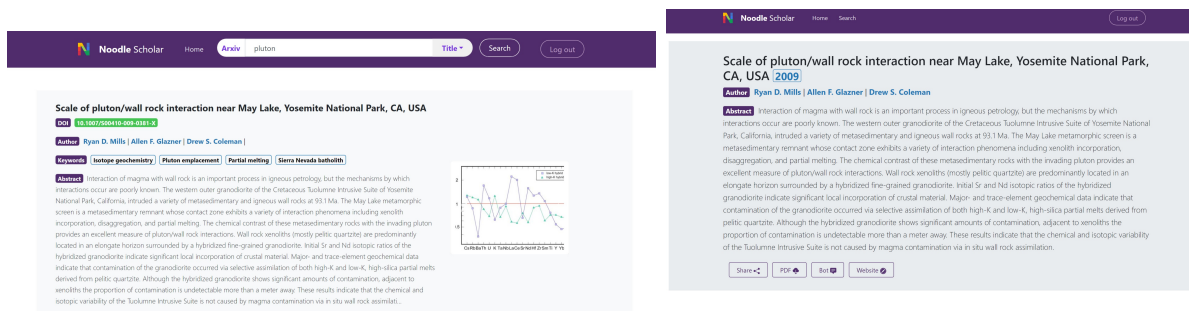
Data Return Format Given the variety of search conditions, we have structured the search body construction process. Depending on the conditions, different keywords or query types are incorporated into the body, significantly reducing code length and enhancing feasibility. Regarding the return of query results, the results are returned as a dictionary. The key of this dictionary is the `hash_id` of the best-matched paper, and its value is another dictionary. This inner dictionary's keys store the matched fields, and the values store the field content, with matched text highlighted on both sides. This structure makes it easier for the backend to process the results.

8 Features

Generally speaking, our design philosophy is **to be close to the actual usage needs of the search engine**. Therefore, **we have focused on designing multiple searches, namely, any potential search terms in any location can be linked to a new page to display search results.**

8.1 Basic Features

1. Support searching by 6 types: All, Title, Abstract, DOI, Keyword, Author
2. Support secondary search in the search-result page. We record the previous page's parameters and make it available inside all pages, by Router.



Everything to be clicked

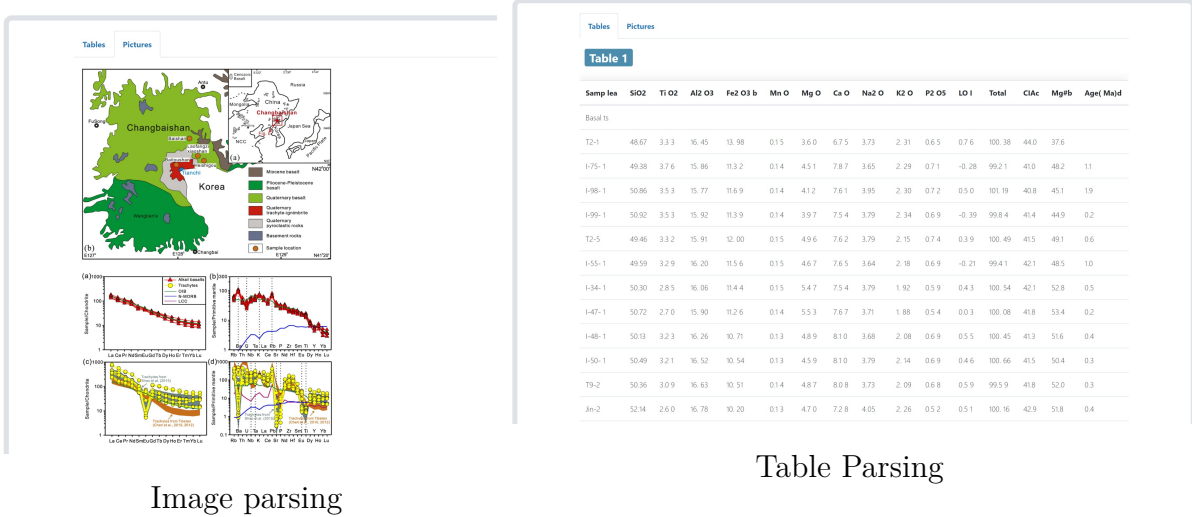
Detailed information supporting triple search

Figure 9: Support secondary and triple search. Preview of image in sliding window

3. Click Author's name or Keyword will route to a new page searching corresponding information, which we name as triple search.
4. Preview of the parsed images in sliding window.
5. Click the DOI to copy DOI. Click Share to copy website's URL. Click Website leads to original web page. Click Download will download PDF, only when user has an account.
6. Provide accurate, completed, powerful table parsing and image parsing.

8.2 Special Features

1. Sufficient database with more than 2,000,000 Arxiv papers, covers numerous fields, such as CS, PHY, STAT, MATH, etc.
2. **Supports special character search.** The search content will be URI-encoded, and further to be decoded at backend, Which means it supports characters like #, %, *space*. The following shows that search engine recognizes the space between authors, then divides several names by space to search.
3. Support LaTeX render.



- Support summary of the paper by GPT model, and further Q&A with any questions related to this paper. It's as well limited to authorized users currently.

8.3 Art

Simple and Non-Mannerism We have unified the design style of all pages. Using purple as the theme color. All elements are FLAT DESIGN.

Adapt Any Screen Size Except for the visualization charts on the homepage, all pages support mobile access, which means they adapt to screens of any size.

Privacy Policy We have drafted a brand new user login privacy agreement to show our emphasis on user privacy.

Promotion Slogan We drafted 6 impressive promotional slogans inspired by Apple Inc.

9 Visualization

We have focused on visualization for the 2,000,000 Arxiv articles, specifically its' Authors and Fields. We display the number of articles each year by pie chart. We display the Top 50 authors and number of their papers. We analyze the proportion of of all included papers'

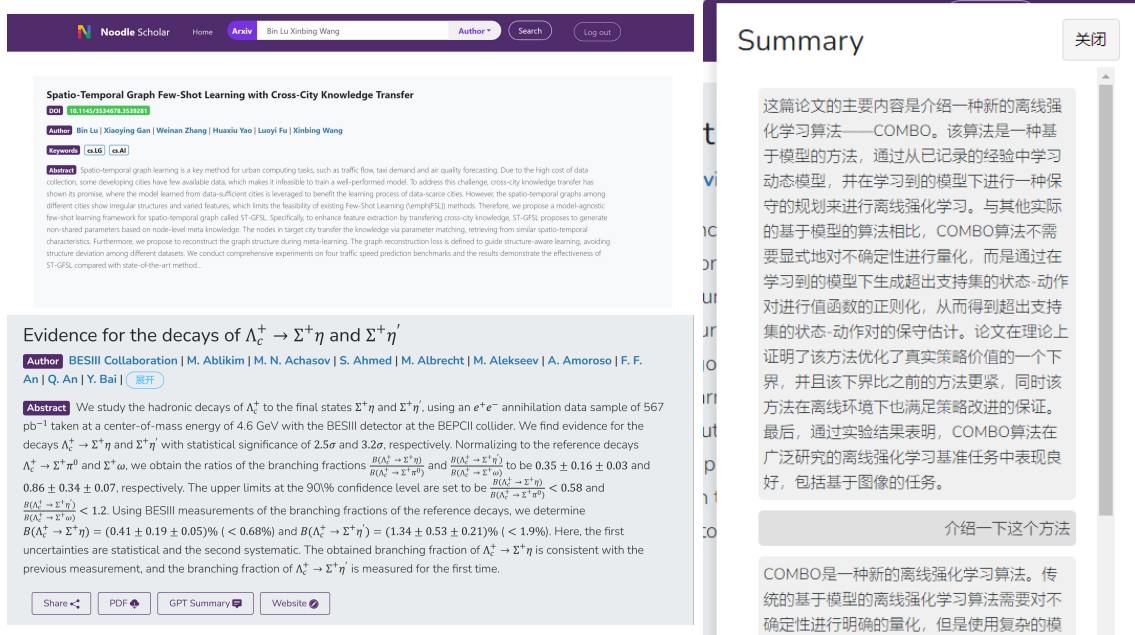
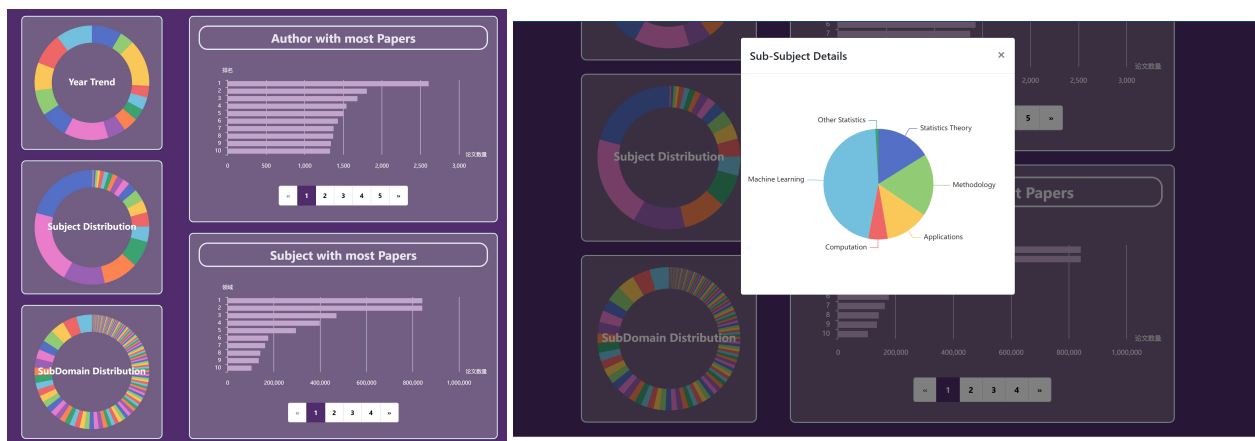


Figure 11: Special Features

fields and sub-fields by pie charts. We analyze the proportion of articles in each sub-field contained in each field by pie charts.

What's more important, each item of these charts, while displaying data, can all be clicked and lead to a new search-result page, showing the search results for that clicked author or field.



Visualization charts

Fields and Sub-Fields connection

Figure 12: Each item of the charts can be clicked to search in detailed

References

- [1] Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. Nougat: Neural optical understanding for academic documents, 2023.
- [2] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- [3] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.